

Metodo delle Correnti di Maglia

Implementazione per la risoluzione di circuiti elettrici

Gianni S., Balliano L., Bianco A.

Politecnico di Torino
Programmazione e Calcolo Scientifico

2025/26

Obiettivo del progetto

- Implementare il **Metodo delle Correnti di Maglia** per risolvere semplici circuiti elettrici.
- Il metodo calcola le **correnti** su ciascun resistore e la relativa **caduta di tensione**.
- Ipotesi semplificative:
 - circuiti formati solo da resistori e generatori ideali di tensione;
 - nessuna resistenza o generatore in parallelo;
 - la netlist descrive un unico circuito (grafo connesso).
- Sfruttando la legge di Ohm $V = RI$, il circuito viene tradotto in un **sistema lineare** risolto con metodi numerici.

Pipeline del metodo

- ① lettura_file: parsing della *netlist* $\rightarrow$ grafo del circuito (m archi, n nodi).
- ② Individuazione delle maglie (due alternative):
 - **DFS + coalbero** (algoritmo_dfs): cicli fondamentali, non minimi;
 - **De Pina** (algoritmo_depina): cicli minimi.
- ③ Costruzione di B , R e del termine noto v (calcola_sistema).
- ④ Risoluzione di $Ai = v$ con $A = B^T RB$ (gradiente coniugato).
- ⑤ Output: tensioni e correnti su ogni resistore (RBi).
netlist $\rightarrow$ grafo $\rightarrow$ maglie $\rightarrow B, R, v \rightarrow Ai = v \rightarrow$ tensioni

Il parser della netlist

- Lettura netlist parametrizzata da riga di comando (argv).
- Lettura **riga per riga** del file di input.
- Il parser scarta in automatico righe malformate, vuote o con intestazioni ingannevoli.
- I dati vengono salvati in due strutture:
 - Una **mappa** circuito: associa la chiave testuale del componente (es. "R1", "V2") ai relativi valori numerici (valore, nodo 1, nodo 2).

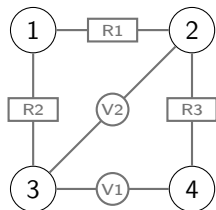
```
std::map<std::string, valor_input> circuito;
```

- Un **vettore** ordine_componenti: salva l'ordine esatto di comparsa originale dei componenti nel file.

```
std::vector<std::string> ordine_componenti;
```

Dal circuito al grafo

- L'assenza di paralleli rende $R1, V2 \dots$ chiavi univoche nella `map`.
- Grafo non orientato; convenzione: arco percorso dal nodo *minore* al *maggiore*.
- La convenzione è imposta dal costruttore di `undirected_edge`: `start` riceve il minimo, `end` il massimo.
- Ogni arco ha così una rappresentazione **canonica univoca**.

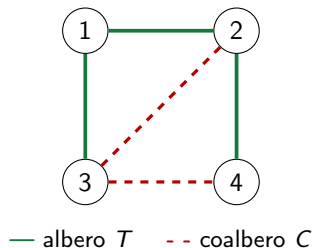


Archi = componenti (R, V)

Nodi = loro capi

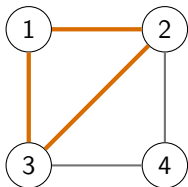
Albero di supporto e coalbero

- Una visita costruisce l'albero T registrando gli *archi di scoperta*.
- Il coalbero è la differenza $C = G \setminus T$.
- La funzione `graph_visit` è generica: se usa uno **stack** fa una DFS, se usa una **coda** fa una BFS.
- $|T| = |V| - 1$, $|C| = |E| - |V| + 1$ (numero di *archi*).
- Ad ogni arco del coalbero C corrisponde una e una sola maglia indipendente.

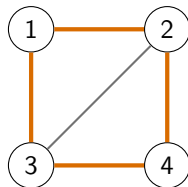


Metodo DFS: dal coalbero alle maglie

Per ogni arco del coalbero si chiude il *ciclo fondamentale* col cammino unico in T .



coalbero (2, 3): cammino $2 \rightarrow 1 \rightarrow 3$
 $\Rightarrow$ ciclo $\{1, 2, 3\}$ (lung. 3)



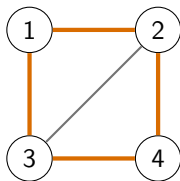
coalbero (3, 4): cammino $3 \rightarrow 1 \rightarrow 2 \rightarrow 4$
 $\Rightarrow$ ciclo $\{1, 2, 4, 3\}$ (lung. 4)

Il cammino in un albero è **unico**: `findpath` lo trova con backtracking.
Cicli corretti ma **non** minimi.

Metodo De Pina: cicli minimi

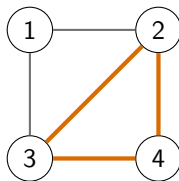
De Pina cerca, ad ogni passo, il **ciclo più corto** non ortogonale al testimone S_i . Sullo stesso grafo, il secondo ciclo trovato è il *triangolo* $\{2, 3, 4\}$, non il quadrilatero della DFS.

DFS: ciclo lungo 4



$\{1, 2, 4, 3\}$

De Pina: ciclo minimo lungo 3



$\{2, 3, 4\}$

Le diverse rappresentazioni dei cicli portano alla stessa soluzione fisica finale (i cicli fondamentali sono linearmente indipendenti).

Perché BFS e non Dijkstra

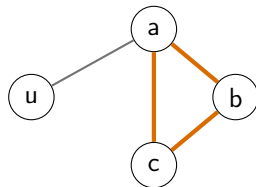
Osservazione chiave

Nel grafo duplicato tutti gli archi hanno **peso unitario**: il cammino minimo è solo questione di *numero di archi*.

- **Dijkstra** è generale (pesi arbitrari): ad ogni passo estrae il nodo col costo minimo da una **coda di priorità** (min-heap). Ogni estrazione e aggiornamento costa $O(\log n)$, per un totale $O(m \log n)$.
- Il fattore $\log n$ è l'**overhead dell'heap**: serve perché i pesi possono essere diversi e il prossimo nodo da chiudere non è prevedibile.
- Con pesi unitari il prossimo nodo è *sempre* quello a distanza più vicina: basta una **coda FIFO**. La **BFS** esplora per livelli e trova il minimo in $O(n + m)$, **senza heap**.
- Usiamo quindi `bfs_cammino_minimo`: stesso risultato di Dijkstra, ma più veloce.

Modulo 2 e grafi limite

- Scelta: niente vettore ausiliario C_μ ; si prende **direttamente** il cammino più corto tra tutti i nodi.
- **Caso limite**: triangolo con una “coda”. Partendo dalla coda, il cammino $u^- \rightarrow u^+$ ripercorre due volte l'arco esterno (lunghezza 5 invece di 3).
- L'algebra mod 2 avrebbe cancellato l'arco doppio; qui il doppio passaggio *allunga* il cammino ...
- ... e il **criterio del minimo lo scarta da solo**, scegliendo correttamente il triangolo.



coda $u-a$ + triangolo $\{a, b, c\}$

Rappresentazione delle Maglie

- Entrambi gli algoritmi (DFS e De Pina) restituiscono i cicli trovati come un **vettore di vettori di nodi in ordine**.

```
std::vector<std::vector<int>>> maglie = algoritmo_depina(res.circuito);  
// std::vector<std::vector<int>>> maglie = algoritmo_dfs(res.circuito);
```

- Il primo e l'ultimo nodo di ogni singolo vettore coincidono sempre, per indicare la chiusura del ciclo (es. 1 2 3 4 1).
- Per agevolare la successiva costruzione delle matrici, la sequenza di nodi di una maglia viene tradotta in **archi ordinati**, salvati come `std::pair (nodo_corrente, nodo_successivo)`.

Costruzione del sistema

La funzione `calcola_sistema` esegue un **singolo ciclo** sul vettore `ordine_componenti`, costruendo R , B , v simultaneamente:

- **Matrice R :** Si posizionano sulla diagonale i valori nominali delle resistenze in Ohm.
- **Matrice B :** Si confronta il verso canonico dell'arco con il `std::pair` della maglia.
 - $+1$ (concorde): resistore percorso nello stesso verso della maglia.
 - -1 (discorde): resistore percorso in verso opposto.
- **Vettore v :** Si valuta l'attraversamento dei generatori per stabilire la caduta di potenziale.
 - Positivo: generatore attraversato dal morsetto $-$ al morsetto $+$.
 - Negativo: generatore attraversato dal morsetto $+$ al morsetto $-$.

Risoluzione: gradiente coniugato

- Il sistema lineare dell'intero circuito diventa $Ai = v$, con $A = B^T RB$.
- La matrice A è **simmetrica e definita positiva** (SPD).
- Il **gradiente coniugato** è il metodo d'elezione per i sistemi SPD: converge in al più n iterazioni.

```
const int it_max = num_maglie + it_extra; // it_extra = 0
const double tol = 1.0e-13; // tolleranza GC
risultato_gc esito = gradiente_coniugato(A, v, it_max, tol);
```

- Dalle correnti di maglia i si ricavano le **correnti fisiche** sui resistori (Bi) e le **tensioni finali** con la legge di Ohm vettoriale (RBi).

Condizionamento

Le maglie minime restituite dall'algoritmo di De Pina rendono A più sparsa e meglio condizionata.

Stampa dei risultati

- Iteriamo su `ordine_componenti`, filtrando ed escludendo i generatori. In questo modo l'output mantiene l'ordine della netlist.
- Poiché anche l'assemblaggio delle matrici in `calcola_sistema` è stato eseguito scorrendo il medesimo vettore, è garantita la corrispondenza biunivoca tra la soluzione numerica (riga in R e B) e il nome del resistore stampato.

Grazie per l'attenzione